SOMAIYA
VIDYAVIHAR

K J Somaiya Institute of Technology
An Autonomous Institute permanently affiliated to University of Mumbai.

DEPARTMENT OF COMPUTER ENGINEERING

"Video Action Recognition : Deep Learning Based Human Activity Classification"

Mini Project Report

by

Jash Sanghvi B2-22
Shriddha Sharma B2-32

Supervisor

Prof. Kavita Bathe

Department of Computer Engineering

K.J. Somaiya Institute of Technology Ayurvihar, Sion Mumbai-400022

2025-2026

INDEX

Abstract

1 Introduction

1.1 Motivation

1.2 Problem Statement

1.3 Objectives

2 Literature Survey

2.1 Survey of Existing System

3 Proposed System

4 Result

5 Conclusion

6 References

Abstract

This mini project presents an end-to-end deep learning framework for human action recognition from video sequences. The primary goal is to classify each input video into one of six predefined categories: CricketShot, JumpRope, Punch, WalkingWithDog, BrushingTeeth, and Unknown. Unlike static image classification, video understanding requires both spatial and temporal learning. Therefore, the proposed system combines a convolutional backbone for frame-wise visual representation and a recurrent network for sequence-level temporal modeling.

The dataset is derived from UCF101 through the Kaggle mirror lifailifai/ucf101. To keep training computationally manageable and reproducible within a mini-project scope, up to 100 videos are sampled per class, resulting in an approximate 600-video subset. Each video is converted into a fixed-length sequence by sampling up to 20 uniformly distributed frames. These frames are resized to 224 x 224, normalized to the [0, 1] range, and passed through MobileNetV2 to obtain compact 1280-dimensional feature vectors per frame.

For temporal learning, a stacked LSTM classifier is trained on the extracted feature sequences using Adam optimization and categorical crossentropy loss. The trained model is saved as lstm_model.keras and integrated into an interactive Streamlit interface for practical local inference. The project also includes Docker and Docker Compose support for platform-independent execution and reproducibility. Inference logic includes confidence-based uncertainty handling through an Unknown threshold, improving practical robustness when predictions are ambiguous. Overall, the work demonstrates a full lifecycle pipeline covering data handling, model training, evaluation, and deployment in a form suitable for academic and prototype-level real-world usage.

CHAPTER 1
INTRODUCTION

Human action recognition is a fundamental and highly active research area in computer vision, with direct applications in smart surveillance, human-computer interaction, sports analytics, autonomous systems, healthcare monitoring, and assistive technologies. In practical scenarios, recognizing an action requires understanding both what is present in individual frames and how visual patterns evolve over time. This makes video understanding significantly more complex than single-image classification.

Traditional frame-wise methods often fail to capture long-term dependencies between motion states. For example, isolated frames of different actions may appear visually similar, while their temporal progression differs substantially. Therefore, robust video classification systems must jointly model spatial appearance and temporal dynamics. This project addresses that requirement through a hybrid transfer-learning pipeline: a pretrained CNN backbone extracts meaningful frame-level features, and an LSTM-based sequence learner models action progression across time.

The selected design offers an effective trade-off between performance, implementation simplicity, and computational efficiency. It avoids expensive end-to-end video models while preserving temporal reasoning through recurrent layers. The resulting framework is modular, interpretable, and practical for academic deployment, making it a strong candidate for mini-project level implementation and evaluation.

1.1 Motivation

Many real-world video intelligence applications require low-cost and practical models rather than only benchmark-optimized architectures. While advanced spatiotemporal models such as 3D CNNs and transformer-based video encoders can deliver strong performance, they often require substantial hardware resources, long training time, and complex deployment workflows. These constraints limit their usability in educational settings and lightweight production environments.

This project is motivated by the need for a compact and deployable action recognition system that can run on standard machines without specialized infrastructure. By leveraging transfer learning through MobileNetV2 and combining it with LSTM-based temporal modeling, the project targets a balanced solution that is easier to train, easier to debug, and easier to integrate into practical interfaces.

Another motivation is reproducibility. The inclusion of a Streamlit frontend for interactive testing and Docker-based runtime support ensures that model behavior can be validated consistently across environments. This aligns with modern engineering expectations where machine learning models are not only trained but also packaged and delivered as usable systems.

1.2 Problem Statement

The problem is to design and implement a deep learning system that can classify short action videos into predefined categories with acceptable accuracy and practical inference speed. The system must:

- process variable-length video inputs,
- convert them into fixed-length model-ready sequences,
- learn temporal patterns across frame features,
- provide deployable inference with confidence-aware Unknown handling.

In addition to core classification, the system should support practical usability constraints. These include robustness to varying video lengths, support for uncertain predictions, and maintainability of the full workflow from preprocessing to deployment. The final solution should be modular enough to support future enhancement, such as additional classes, threshold tuning, and more advanced temporal architectures.

1.3 Objectives

- Build a six-class action recognition pipeline using UCF101-derived data.
- Use MobileNetV2 as a frame feature extractor and LSTM as a temporal classifier.
- Train using a reproducible notebook workflow and persist reusable artifacts.
- Provide local UI inference and Docker-compatible execution.
- Include uncertainty handling through a configurable confidence threshold.

Secondary objectives are also considered to strengthen the engineering quality of the project:

- maintain clear separation between training, evaluation, and inference components,
- keep preprocessing deterministic for fair repeatable experiments,
- preserve trained models and intermediate artifacts for rapid re-evaluation,
- create a project structure that can be extended for future coursework or research prototypes.

CHAPTER 2
LITERATURE SURVEY

Human action recognition research has evolved through multiple methodological generations. Early methods relied on handcrafted features such as optical flow descriptors, motion history images, and trajectory-based representations. Although informative in controlled settings, these techniques were sensitive to illumination changes, camera motion, viewpoint variation, and background clutter.

With the emergence of deep learning, CNN-based approaches significantly improved feature robustness by automatically learning hierarchical visual representations. Contemporary systems generally follow one of three broad categories: frame-wise 2D CNN aggregation, recurrent sequence modeling over deep features, and end-to-end spatiotemporal learning through 3D CNNs or attention-based transformers. Each category offers different trade-offs in accuracy, data requirements, and computational cost.

Recurrent models such as LSTM remain widely used in resource-constrained contexts because they allow temporal learning over compact feature vectors extracted by pretrained CNN backbones. This decoupled design reduces computational burden while retaining sequence-awareness, making it suitable for practical and educational deployments.

2.1 Survey of Existing Systems

Traditional handcrafted approaches based on optical flow and trajectory descriptors are computationally lighter but generally less robust than deep methods, especially under complex backgrounds and inter-class visual similarity. These systems also require manual feature engineering, which limits adaptability when action classes change.

End-to-end 3D CNN methods learn joint spatial-temporal representations directly from raw clips and often produce strong benchmark performance. However, they typically require large labeled datasets, high memory consumption, and longer training cycles. This makes them less practical for mini-project timelines and limited hardware environments.

A practical middle path is to combine pretrained 2D CNN embeddings with sequence models. In this strategy, each frame is first encoded by a strong visual backbone such as MobileNetV2, then recurrent layers aggregate temporal dependencies across the frame sequence. This hybrid method is more data-efficient and easier to deploy while still capturing critical action dynamics.

The proposed project follows this transfer-learning-centered approach. It aligns with current applied machine learning practices where pretrained backbones are reused for domain tasks, reducing the training burden and improving reproducibility.

CHAPTER 3
PROPOSED SYSTEM

The implemented system follows a staged pipeline:

1. Dataset preparation
- Source: Kaggle UCF101 mirror (lifailifai/ucf101)
- Selected classes: CricketShot, JumpRope, Punch, WalkingWithDog, BrushingTeeth, Unknown
- Sampling: up to 100 videos per class

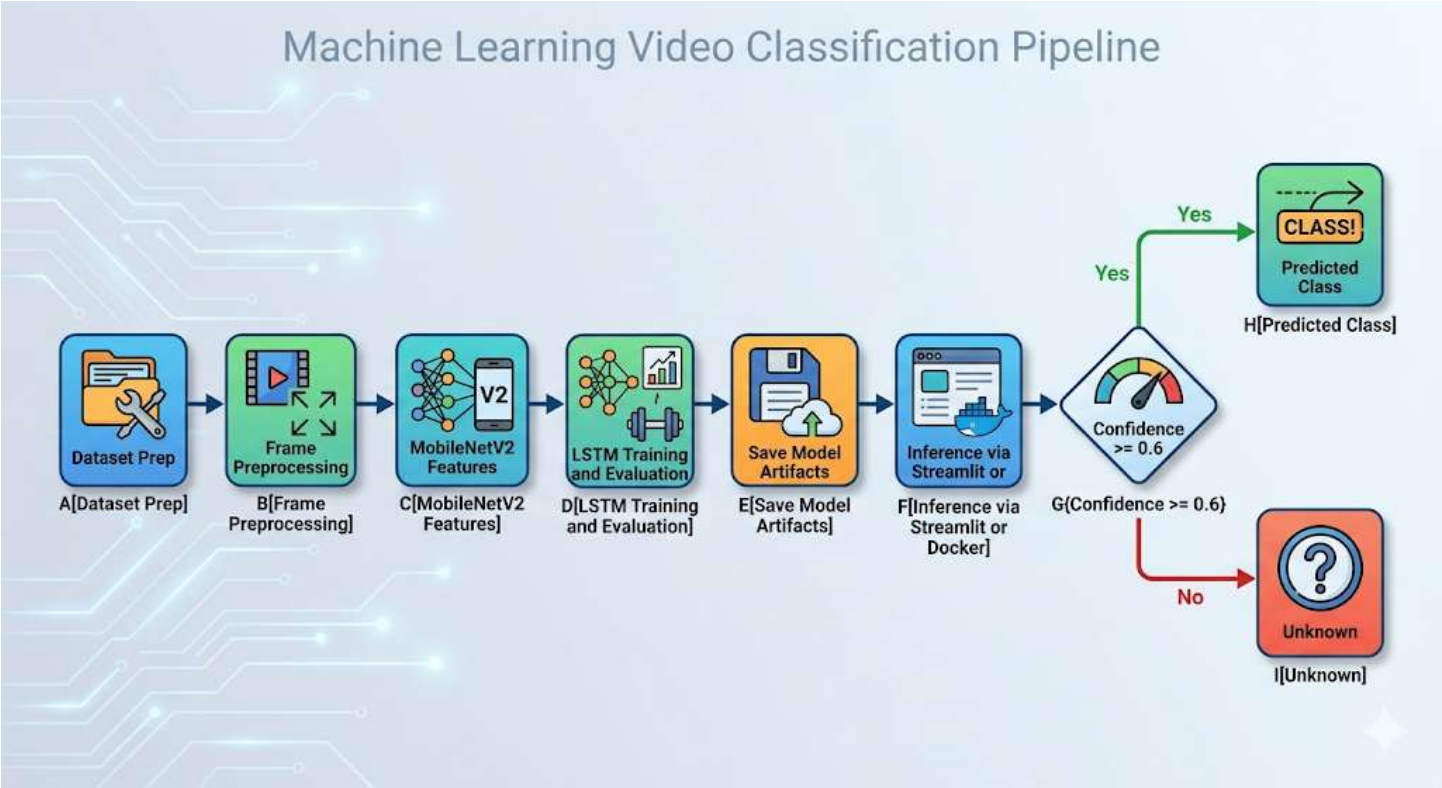
2. Video preprocessing
- Uniformly sample up to 20 frames per video
- Resize each frame to 224 x 224
- Normalize pixel values to [0, 1]

3. Feature extraction
- Backbone: MobileNetV2 (include_top=False, pooling='avg')
- Output: 1280-dimensional feature vector per frame
- Sequence tensor shape per sample: (20, 1280)

4. Sequence classification
- LSTM(128, return_sequences=True)
- Dropout(0.3)
- LSTM(64)
- Dropout(0.3)
- Dense(32, relu)
- Dense(6, softmax)

5. Training configuration
- Split: 80/20 stratified train-test
- Optimizer: Adam
- Loss: Categorical crossentropy
- Metric: Accuracy
- Epochs: 50
- Batch size: 8

6. Deployment
- Streamlit app for local interactive inference
- Docker and docker-compose for reproducible runtime
- Unknown/uncertain detection based on confidence threshold (default 0.6)


Machine Learning Video Classification Pipeline
Dataset Prep
A[Dataset Prep]
Frame Preprocessing
B[Frame Preprocessing]
V2
MobileNetV2 Features
C[MobileNetV2 Features]
LSTM Training and Evaluation
D[LSTM Training and Evaluation]
Save Model Artifacts
E[Save Model Artifacts]
Inference via Streamlit or
F[Inference via Streamlit or Docker]
Confidence >= 0.6
G{Confidence >= 0.6}
Yes
Yes
CLASS!
Predicted Class
H[Predicted Class]
No
Unknown
I[Unknown]

CHAPTER 4
RESULTS

The system successfully trains and performs multiclass inference for the target actions, confirming the correctness of the end-to-end workflow. Evaluation in the notebook includes test-set accuracy through model.evaluate and class-wise precision, recall, and F1-score through classification_report. These metrics provide both aggregate and class-level visibility into model behavior.

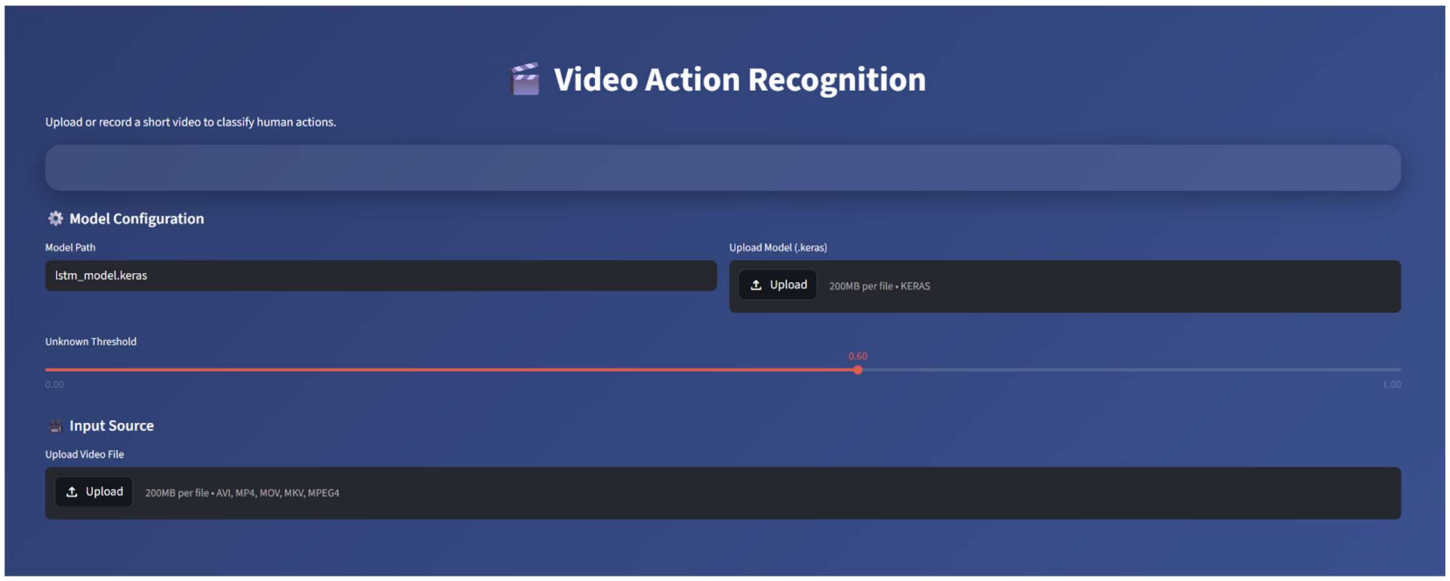
From a qualitative perspective, the model demonstrates the ability to capture action-specific temporal patterns across sampled frames. The stacked LSTM layers effectively utilize ordered feature embeddings, indicating that temporal sequencing contributes meaningful signal beyond frame-level appearance alone.

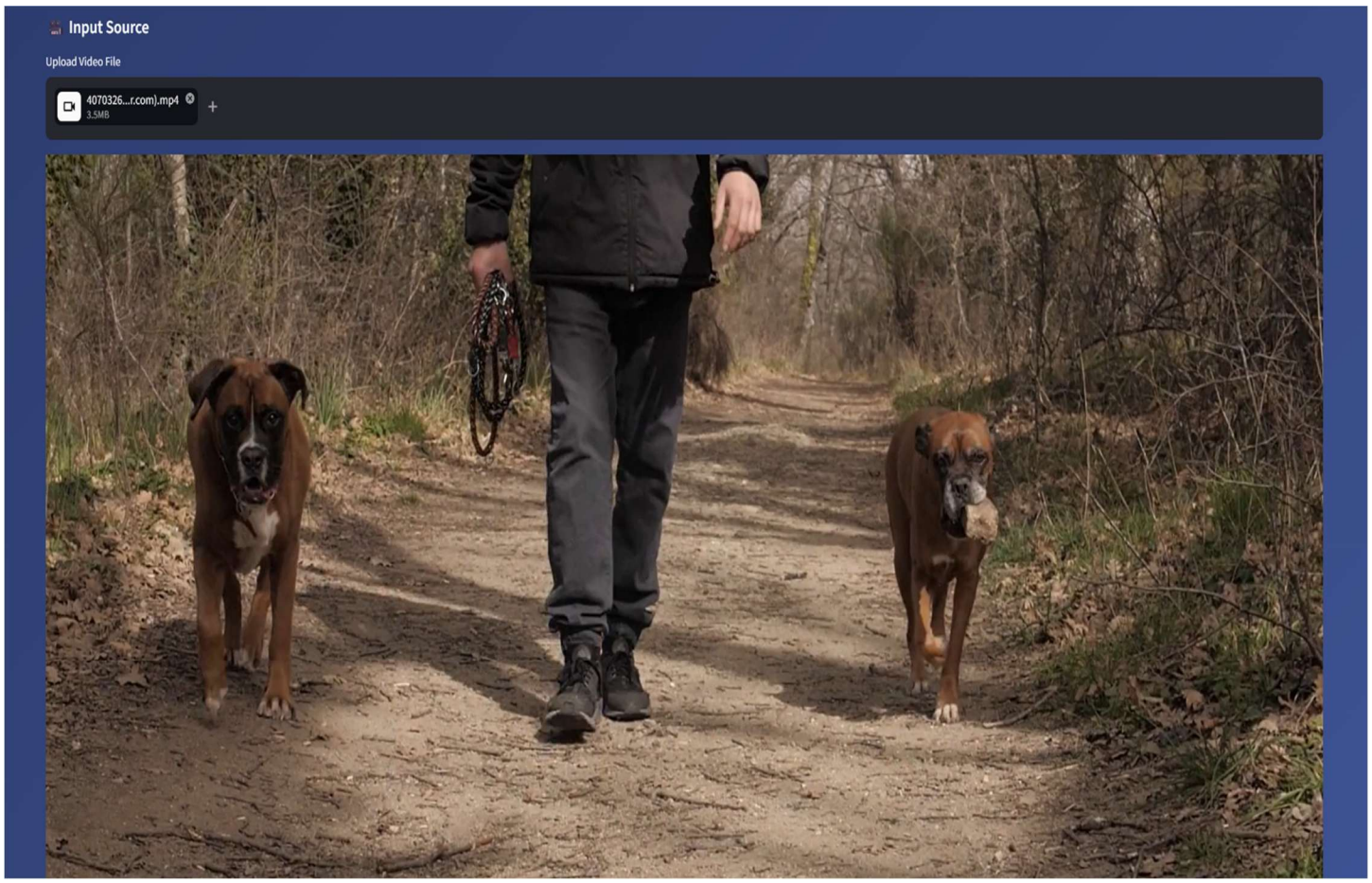
The project also validates uncertain-input handling through threshold-based Unknown logic. During inference, the model computes a probability distribution across all classes. If the highest confidence is below a configured threshold, the prediction is flagged as Unknown or Uncertain. This improves practical usability, especially in open-set conditions where unseen actions or noisy videos may be provided.

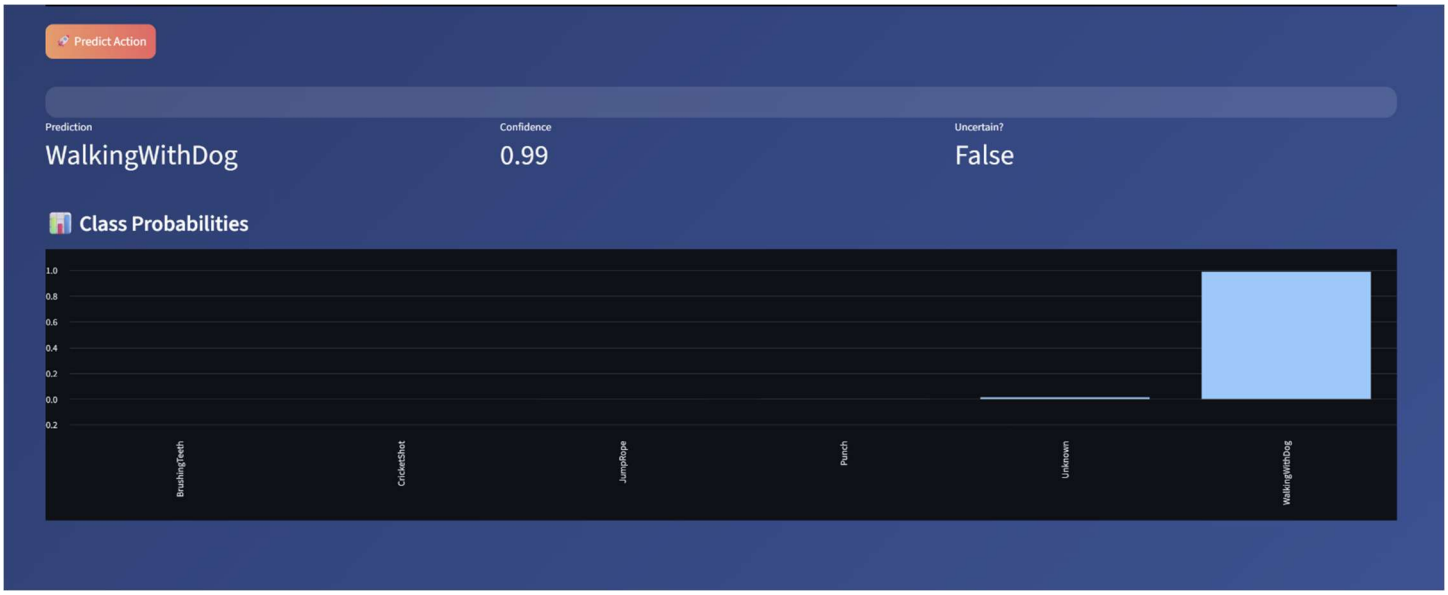
In deployment terms, the Streamlit interface allows direct user interaction with uploaded videos and displays class probability outputs, making model behavior transparent. Docker support adds operational consistency and reduces environment-specific issues, which is valuable for demonstrations, evaluation, and collaborative testing.

Generated artifacts include:
- lstm_model.keras
- features.npy
- labels.npy

Supporting deployment files include:
- app.py (Streamlit UI)
- Dockerfile
- docker-compose.yml
- requirements.txt
- DOCKER.md and LOCAL_UI.md guides


Video Action Recognition
Upload or record a short video to classify human actions.
Model Configuration
Model Path
lstm_model.keras
Upload Model (.keras)
Upload
200MB per file • KERAS
Unknown Threshold
0.60
0.00
1.00
Input Source
Upload Video File
Upload
200MB per file • AVI, MP4, MOV, MKV, MPEG4


Input Source
Upload Video File
4070326...r.com).mp4
3.5MB



Screenshots of the website that was built

CHAPTER 5
CONCLUSION

recognition. By combining MobileNetV2-based frame embeddings with LSTM temporal modeling, the system captures both spatial semantics and sequential motion patterns in a computationally efficient manner.

The project meets its primary educational and engineering goals: dataset preparation, deterministic preprocessing, sequence model training, evaluation through standard classification metrics, and practical inference through both local and containerized execution paths. The resulting system is not only functional but also extensible, which is important for future coursework, benchmarking, and prototype evolution.

Beyond technical implementation, the project highlights a key machine learning systems principle: accuracy alone is insufficient without usability and reproducibility. The inclusion of Streamlit for human-facing interaction and Docker-based packaging strengthens the solution from an academic model into a practical software artifact.

Future work can improve performance and reliability through confidence threshold calibration, richer error analysis (including confusion matrices), dedicated train-validation-test partitioning, class balancing strategies, temporal data augmentation, and experimentation with alternative sequence architectures such as GRU, bidirectional recurrent models, temporal convolutional networks, or transformer encoders.

CHAPTER 6
REFERENCES

Barczak, A.L.C., et al. (2011). "A New Modified ASL Database for Hand Gesture Recognition Systems." *Proceedings of the International Conference on Image and Vision Computing New Zealand (IVCNZ).*

Cheok, M.J., et al. (2019). "A Review of Hand Gesture and Sign Language Recognition Techniques." *International Journal of Machine Learning and Cybernetics, 10(1), 131–153.*

Krizhevsky, A., Sutskever, I., & Hinton, G.E. (2017). "ImageNet Classification with Deep Convolutional Neural Networks." *Communications of the ACM, 60(6), 84–90.*

Lugaresi, C., et al. (2019). "MediaPipe: A Framework for Building Perception Pipelines." *arXiv preprint arXiv:1906.08172.*

Pigou, L., et al. (2018). "Beyond Temporal Pooling: Recurrence and Temporal Convolutions for Gesture Recognition in Video." *International Journal of Computer Vision, 126(2–4), 430–439.*

Taskiran, M., et al. (2020). "Real-Time Sign Language Recognition Using CNN and Optimization Techniques." *Multimedia Tools and Applications, 79(21), 15275–1529*